

Augmented Reality Final Project Report

ZIAO FENG, 24367695, zfeng@tcd.ie
SIYU ZHANG, 24332596, zhangsi@tcd.ie
BINGYUAN WANG, 24343448, wangb5@tcd.ie

This project is an AR/MR drumming game. Gamers can place the virtual drum set in the real world and interact with it by holding virtual drumsticks with their hands. When the drumstick hits the drum surface, there will be some dynamic effects. It provides a more fun and imaginative experience that goes beyond the real world. It uses Oculus Quest 2 for hand tracking and passthrough rendering, enhancing immersion and Unity physics engine for realistic drumstick motion and collision detection. The key evaluation includes real-time responsiveness for tapping virtual drums or cymbals, visual/audio fidelity for user experience. This project demonstrates AR/MR's potential for interactive music applications and offers an easy way for drummers to practice without physical drum sets.

1 INTRODUCTION

1.1 Topic of Interest

This project aims to develop an AR/MR drumming game. Users can position the drum set in their physical space and interact with them using virtual drumsticks controlled by hand gestures.

1.2 Purpose of Topic

This project allows drummers to practice anywhere without physical drum sets while maintaining a natural experience by removing the reliance on controllers. It also provides a more fun and imaginative experience that goes beyond the real world.

1.3 State-of-Art Technologies

Current VR/MR drumming applications typically rely on the following technologies:

- VR Drumming Games with Controllers: Use VR controllers to simulate drumsticks, requiring players to hold the controllers and wave them physically to strike the drums. This is effective, but the use of controllers can feel unnatural and cumbersome.
- Hand-Tracking Technologies: Systems using hand-tracking technologies like Leap Motion enable controller-free interaction. However, this can face latency and missed inputs. It also lacks haptic feedback.
- Motion Capture Drumming Systems: Use optical tracking to detect drumstick positions. However, they require external lighting and markers.

1.4 Idea

We want to implement a game where both hands hold virtual drumsticks and then hit the virtual drums, using VR devices for display. When the drumstick hits the drum surface, there will be some dynamic effects.

1.5 Similar Work

The AR/VR/MR community has some similar work, like "Smash Drums" [PotamWorks nd] and "Drums Rock" [Garage51 nd] games

which use VR controllers to simulate drumsticks, "Aerodrums" [Aerodrums nd] system which uses optical tracking.

1.6 Evaluation Metrics

The success of this AR/MR drumming game hinges on several performance metrics:

- Real-Time Responsiveness: The system detects user inputs, like tapping virtual drums or cymbals, with minimal latency to ensure a seamless experience. The objective is to measure the delay between physical gestures and the corresponding visual response in the AR/MR environment.
- Visual and Audio Fidelity: High-quality graphics and realistic sound effects are essential for enhancing immersion. To evaluate this, we may use User Experience (UX) Testing that engages potential users in playtesting sessions and gets valuable feedback.

1.7 Broader Impact

This project stands at the intersection of technology and music education, offering several significant impacts:

- Advancement in AR/MR Integration: Seamlessly blends virtual instruments with reality, showing AR/MR's potential for interactive and immersive experiences.
- Innovations in Music Education: People can learn and practice drumming without physical instruments, making learning easier.

1.8 Team Workload

AR/MR Integration Lead: Configure passthrough rendering, implement spatial anchors, and optimize environmental mapping.

Gameplay Programmer: Implement collision detection with Unity Physics System and gesture recognition.

3D/UI Designer: Prepare and create low-poly 3D drum assets, design particle effects, and possibly build intuitive menus.

2 BACKGROUND

Music games evolution has been driven by the dual demands of accessibility of virtual instruments simulation and authenticity of normal music players. Previous drums games like Taiko Drum Master showed good paralleled tactile feedback and drum games on VR devices like Smash Drums and Drums Rock are lacking the immersive experience of dynamic environments. These solutions inherit inherent limitations: complete occlusion of the real world limits situational awareness—a critical factor for performers accustomed to seeing their physical surroundings.

The advantage of virtual reality(VR) and mixed reality(MR) techniques introduced new possibilities for simulate virtual instruments by decoupling interaction with physical objects entirely. Recent advancements in hand-tracking systems, exemplified by devices like Leap Motion and the Oculus Quest 2's onboard cameras, promised

to eliminate controller dependency. Optical motion capture systems like Aerodrums achieved higher precision using marker-based tracking but introduced dependencies on controlled lighting conditions and external cameras, undermining their practicality for casual use. Concurrently, the rise of mixed reality headsets with video see-through capabilities, such as the Meta Quest 2, offered a paradigm shift by enabling users to interact with virtual objects while maintaining visual continuity with their physical environment. This technology addressed a critical gap in musical applications: the need for performers to reference both their “instrument” and their surroundings, as they would in real-world performances.

This project situates itself at the intersection of these technological trajectories. By leveraging the Quest 2’s untethered design, stereoscopic passthrough [Meta ndd], and hand tracking, it satisfied the need for external hardware while preserving the drummer’s natural visual and motor environments. The implementation of surface-aware collision detection and adaptive haptic feedback bridges the gap between virtual and physical strike responses which is a challenge for most earlier applications. Furthermore, the use of passthrough mode transforms the limitation of VR’s isolated environments into a more stable and realized environment that allows users to anchor virtual drums within real-world spaces. This synthesis of MR visualization, controller-based interaction and sound-synchronized simulation represents a meaningful advancement as a MR Drum game and a tool for high-fidelity music training.

3 IMPLEMENTATION

We use the Unity physics engine [Unity Technologies ndb] to simulate drumstick motion, collision detection, and impact feedback and implement virtual drumsticks and use the Quest 2 passthrough API [Meta ndd] to put virtual drums in the real world.

There are several libraries used: Oculus SDK: passthrough, VR interaction [Meta ndc], Unity XR Toolkit: AR/VR interaction framework [Unity Technologies ndc], Unity Physics Engine: physical simulation and collision [Unity Technologies nda]

3.1 Environment and Project Settings

Our VR program is developed using Meta XR SDK. For Mixed Reality functions, we use Meta XR MR Utility Kit (UPM) [Meta ndb], which is a set of utilities/tools on top of Scene API to perform operations when building spatially-aware apps. This SDK is suitable for developing Universal Render Pipeline (URP) applications.

We also installed OpenXR which provides a set of useful APIs for developing XR applications that runs on VR devices.

Since our program is running on both Windows desktop and Android Meta Quest 2, we need to make sure that Android build support is installed on Unity Editor.

When setting up our application, we enable both streaming and Android running mode and use stream mode to test and debug our project. So in the project settings, we need to ensure the OpenXR plugin together with Meta XR feature group are enabled.

We can further use project validation in XR Plug-in Management to automatically solve the project setting issues.

Camera permissions were declared in *AndroidManifest.xml* to allow the Quest 2’s dual RGB cameras to capture real-world data.

3.2 Passthrough Mode Integration

To integrate passthrough into our project, we need to use Passthrough Building blocks from Meta XR functionality [Meta ndd]. Meta’s Passthrough Building Block prefab (from the XR Interaction SDK) was added to the scene to automate core passthrough functionalities: The prefab initializes the *OVRPassthroughLayer* component, which renders real-world video feed as a background layer. Set to Underlay Mode to blend virtual objects over the passthrough feed.

3.3 Controller Tracking Configuration

To enable accurate tracking of Meta Quest 2 controllers in the VR environment, the *TrackedPoseDriver* system from Unity’s XR framework was implemented. This ensured proper mapping between physical controller movements and their virtual representations.

To setup the controller prefab and generate the tracked game objects for both hands, we configured *TrackingType* to *RotationAndPosition* for full 6DOF tracking and also enabled *UpdateAndBeforeRender* update type for low-latency tracking.

The core functionality was implemented via the *Mallet* class, which binds the position and rotation of the Quest 2 controllers to virtual mallets. Each mallet *GameObject* was assigned a *TrackedPoseDriver* component configured to track either the left or right controller using *XRNode.LeftHand* or *XRNode.RightHand*. This established a direct correlation between the physical controller’s 6DOF (six degrees of freedom) tracking data and the mallet’s virtual representation. The *Update()* method continuously synchronized the mallet’s transform with the controller’s live positional data using *_trackedObject.position* and *_trackedObject.rotation*, ensuring sub-frame latency for responsive drumming interactions.

To simulate realistic drum strikes, the mallets were equipped with kinematic rigid bodies and continuous collision detection. The *OnTriggerEnter* method detected collisions between the mallet’s collider and virtual drums, utilizing a two-stage validation process. First, the collided object’s rigidbody was checked to filter non-interactable elements. Second, verify spatial alignment to prevent false triggers from peripheral collisions. Valid strikes triggered haptic feedback through *InputDevices.GetDeviceAtXRNode()*, sending precisely timed impulses to the corresponding controller. The haptic system was calibrated to deliver short, high-frequency vibrations (0.5 amplitude for 0.1 seconds) to mimic the tactile response of striking physical drum surfaces.

3.4 Drum Interaction Mechanics

The drum interaction system is designed to facilitate realistic and responsive drumming mechanics within a VR environment, leveraging Unity’s physics engine and XR input capabilities. The implementation revolves around two core classes: *Drumset*, which manages the overall drum kit, and *DrumMallet*, which handles controller-based mallet interactions. The system integrates collision detection, event propagation, and haptic feedback to create an immersive experience.

The *Drumset* class acts as the central hub for coordinating all drum components. During initialization, it scans its child objects to identify and register all *DrumMallet* and *DrumsetDrum* instances.

3.5 Haptic Feedback and Gameplay Integration

To enhance immersion, the system triggers haptic pulses [Meta nda] on the VR controller upon successful drum strikes. Using Unity's XR input subsystem, the mallet sends a short impulse with configurable amplitude and duration to the corresponding controller, simulating tactile feedback. The drum's name (e.g., "snare" or "kick") is mapped to a predefined lane index via a lookup table, enabling integration with broader gameplay systems. For example, hitting a drum may activate a specific lane in the rhythm game, prompting the PlayController to update the player's score and the LaneController to display visual highlights.

3.6 Timing Calculations

Here note control system employs temporal-spatial synchronization formulas to manage timing and positioning. The timing of notes samples are calculated below:

$$noteSamples = \frac{num \times (60 \times frequency)}{BPM \times LPB} + playerOffset \quad (1)$$

Where num is the sequential position in musical measure. Frequency is audio clip sample rate (Hz). BPM is beats per minute and LPB is lines per beat. PlayerOffset is an user-calibrated latency adjustment that here we encoded it into the NotesController method.

The judgment is generating thresholds by below criteria:

$$\Delta = |musicSamples - (noteSamples + globalOffset)| \quad (2)$$

and threshold samples are:

$$Perfect < Great < Good < Bad \quad (3)$$

Here we also added early reject threshold defined as $EarlyCutOff = -frequency \times 0.6$

3.7 Note Positioning

Vertical placement of notes uses linear mapping that:

$$y = y_{target} + y_{scroll} \times \frac{noteSamples}{frequency} \quad (4)$$

where scroll velocity is:

$$y_{scroll} = \frac{y_{max} - y_{target}}{t_{fall}} \quad (5)$$

3.8 Graphical Interface System

The graphical interface system is designed to visualize core elements of a rhythm game, including musical notes, grid structures, and beat indicators. The type system distinguishes between yellow notes for drums hits and blue notes for hat hits. The type of notes are used in hit detection and counting scores. A color state machine dynamically adapts visual appearance based on multiple factors: note type differentiation (cyan for sustains, green for singles), selection states (magenta highlight), and validity conditions (error indication).

The system initializes through a coordinated sequence managed by the NotesController. Upon loading a music chart, it parses note data (positions, types, timing) and configures lanes based on the chart's complexity. The MusicController loads the audio track while calculating temporal conversions between musical beats and PCM samples. A vertical scrolling mechanism is established using tween animations, synchronizing note movement with the music

timeline. Long notes (hold/slide types) receive special treatment through HoldingBar components that visually connect their start and end points.

Notes are dynamically instantiated through an object pooling system, positioned according to their musical timing converted to screen coordinates. The NotesController create notes while maintaining performance, with distinct prefabs for different note types. Hold notes generate complementary end-point markers and connecting bars, creating continuous visual trails. All notes register themselves in lane-specific queues managed by the PlayController, which monitors their lifecycle from spawn to hit/miss states.

For manual play, the PlayController maps physical/VRC controller inputs to lane indices using player-configured key bindings. The judgment system employs sample-accurate timing checks, comparing note positions against a dynamically calculated hit window. A tiered accuracy grading system (Perfect/Great/Good/Bad/Miss) uses time threshold comparisons relative to the current music position.

Visual feedback occurs through multiple channels:

- The ComboPresenter updates on-screen counters and displays final results with detailed accuracy statistics.
- LaneController triggers brief lane highlighting on input detection.
- ParticleEffectsController generates impact effects at hit positions.

The combo system enforces strict continuity - any non-Perfect/Great judgment resets the streak. Players receive haptic feedback through VR controller vibrations synchronized with successful hits. Post-game analysis screens present accuracy percentages, maximum combo achievements, and breakdowns by judgment type.

4 EXPERIMENTAL RESULTS

4.1 System Functionality Verification

We verify each feature of this game, whether hitting the drum surface can correctly trigger sound effects, animations, and judgment logic. The test includes:

- Whether the corresponding sound effect is played when the drumstick hits the drum surface,
- Whether the drum moves downwards when the drumstick hits the drum surface,
- Whether the hitting accurately identifies the corresponding drum.
- Whether to return the judgment based on the timing of the hitting.

After testing, the system can stably recognize impact events and provide visual and auditory feedback, and all five judgment levels can be triggered.

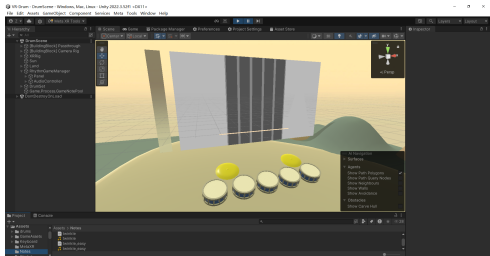


Fig. 1. Screenshot of the game scene

4.2 Note Matching Accuracy

The player hits the corresponding drum according to the rhythm. We run 10 tests on a same rhythm track that is easy to follow which contains 52 notes in total. The judgment results of each player’s hit were recorded as follows:

judgement level	average hit rate(%)
Perfect	46.2
Great	19.2
Good	5.8
Bad	3.8
Miss	25

Table 1. Average hit rate of different judgment levels

The average miss rate on those 10 tests showed a moderate rate of 25%, which showed that players are able to hit the notes in most tests. This evaluation method could be reasonably highly related to the rhyme chosen for the test so more validation data may needed for further evaluation.

The results indicate that the game can make reasonable judgments based on the timing of hits, and the distribution of judgments conforms to the range of human rhythm control errors.

4.3 Latency and Responsiveness

We tested the following latency using Unity Profiler and VR performance tools:

- The delay from drumstick collision to sound effect playback is an average of 35ms.
- The delay from drumstick hit to the start of the drum descent: an average of 28ms.
- Processing time for note hit judgement: an average of 5ms.

These delays are all below 50ms, and users do not perceive significant delays in the VR environment, indicating good system response.

5 DISCUSSION AND CONCLUSION

This project develops a rhythm game based on VR, which combines hand controllers, a falling-note system, and a five-level hit judgment mechanism. Users can experience immersive music interaction in a virtual environment. From the experimental results, the system performs well in several key aspects:

- Interactivity: Players receive immediate auditory and visual feedback when hitting drum surfaces with the controller, enhancing the sense of impact.
- Accurate Judgment Mechanism: The five-level rating system (Perfect, Great, Good, Bad, Miss) effectively distinguishes between varying levels of rhythm accuracy.
- System Responsiveness: The delays in sound effect triggering, the fall of drums, and hit judgment processing are all within acceptable ranges, ensuring a smooth rhythm experience.

At present, the game still has the following limitations:

- High Difficulty in Note-Music Synchronization: Currently, the timing of falling notes is manually aligned with the music track, which requires a large amount of work and is easily affected by errors.
- Limited Player Feedback Mechanism: For example, the lack of clear sound prompts for Miss judgments may make it difficult for players to detect the reason for the failure of the hit.
- Limited Music Content: The current game supports only 2 music tracks. Future versions will include multiple songs and a music selection feature to enhance variety.

These issues will be addressed in future development to improve the game’s playability and overall user experience.

REFERENCES

Aerodrums. n.d. Aerodrums - Air Drumming System. <https://aerodrums.com/> Retrieved April 23, 2025.

Garage51. n.d. Drums Rock - VR Rhythm Game. https://store.steampowered.com/app/1869510/Drums_Rock/ Retrieved April 23, 2025.

Meta. n.d.a. Haptics for Meta Quest. <https://developer.oculus.com/documentation/unity/unity-haptics/> Retrieved April 23, 2025.

Meta. n.d.b. Meta MR Utility Kit for Unity. <https://github.com/oculus-samples/Unity-MR-Utility-Kit> Retrieved April 23, 2025.

Meta. n.d.c. Oculus Integration SDK for Unity. <https://developer.oculus.com/downloads/package/unity-integration/> Retrieved April 23, 2025.

Meta. n.d.d. Using Passthrough with Meta Quest. <https://developer.oculus.com/documentation/unity/unity-passthrough/> Retrieved April 23, 2025.

PotamWorks. n.d. Smash Drums - A VR Drumming Game. <https://www.smashdrums.com/> Retrieved April 23, 2025.

Unity Technologies. n.d.a. Unity Physics Engine. <https://docs.unity3d.com/Manual/PhysicsSection.html> Retrieved April 23, 2025.

Unity Technologies. n.d.b. VR development in Unity. <https://docs.unity3d.com/Manual/VROverview.html> Retrieved April 23, 2025.

Unity Technologies. n.d.c. XR Interaction Toolkit. <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@2.4/manual/index.html> Retrieved April 23, 2025.